Setup structure (config, JSON file)

required fields:

gameInitOptions: a description of the teams and roles in the game. role presets and additional general settings

gameLoop: a description of the game logic, consisting of a sequence of actions.

winCondition: a description of the winning conditions of teams.

optional fields:

visualSettings: front visual effects settings that have no relation to the game logic

beforeLoopActions: actions performed once at the beginning of the game

turnSpectatorToPlayerActions: actions performed when a spectator turns into a player

turnPlayerToSpectatorActions: actions performed when a player turns into a spectator

postGameActions: actions that are not part of the logic of the game and are performed after the game ends.

gameInitOptions

required fields:

teams: record like { teamId (string) :
{
id: string; (same as teamId)
name: string;
description: string;
color: string; (as CSS value)
roles: Array of roleIds (strings);
},
…}

roles: Array of record {
roleInfo: {
id: string;
name: string;
description: string;
avatar: string; (URI)
team: string; (teamId)
prefix?: string; (is added before the name in the role confirmation)

};
isRequired: boolean; (if ‘true’ the role must be in the game)
isDefaultRole?: boolean; (if ‘true’ the role will be used by default)
}

rolesPreset: record like
{
presetName (string): {
qnt: number; (number of players)
presets: array or records
{
roleId (string): number;
}
},
…
}

optional fields:

roleConfirmation: boolean. if ‘true’ at the beginning of the game all players will be asked to confirm their roles
roleConfirmationSound: string; (URI or sound alias. If set, plays during role confirmation)

lockedRoles: Array of strings; (role ids) Setting that allows to assign a certain role (roles) included by the user in the game to a player even if there are additional roles. For example, if there are 13 players in the game and the user chooses 15 roles, 3 of which are lockedRole ‘mafia’ - all 3 mafia must be guaranteed to be in the game.

extraCards: boolean; If ‘true’ unallocated roles become additional cards in the game.

voices: record {
libraryName (string):
{soundName (string): string (URI), …},
…
}
soundboard: record {
libraryName (string):
{soundName (string): string (URI), …},
…
}
images: record {
imageName (string): {
url: string
tooltip?: string },
…},
animations: record {
animationName (string): string (lottie URI),

…},

minPlayers: number;
maxPlayers: number;
minExtraRoles: number;
maxExtraRoles: number;
useDefaultRoles: boolean;
welcomeSound: string; (URI)
setRolesByGame: boolean; (if 'true' roles are assigned by game logic and role confirmation is skipped)
timePerRound: number; (in seconds)
configVariables: **(**variables that can be configured in the game settings wizard. Its saved in game cache)

array of records:
{
name: string;
label: string;
type: string; ('number', 'string', 'boolean', 'list')
default: number | string | boolean;
range_min?: number;
range_max?: number;
values?: string[];
}

allowSpectatorBecomePlayer: boolean

allowPlayerBecomeSpectator: boolean
removePlayerTimer: number; (in seconds. Default: 180)

notChangeLayoutAfterGame: boolean (true by default);

allowRecorder: boolean (if 'true' host can request recording permission)
spectatorsCanHearBreakoutRooms: boolean (if 'true' spectators can hear breakout rooms; true by default)

cheatsheet: {"label": [label], "image": [image url]} If present, a button appears in the bottom right. The button has the label [label], and when you click the button, it displays the image at [image url] as a popup.

preferredPlayersQnt: array of numbers;

[DEPRECATED] please use visualSettings section for this fields:
showImageRowOnSmallScreen: boolean; (true by default);
isCardAnimationsOff: boolean; (false by default);
increaseHandHeight: boolean

visualSettings

all fields are optional

showImageRowOnSmallScreen: boolean; (true by default);
isCardAnimationsOff: boolean; (false by default);
increaseHandHeight: boolean;
cardHandBackgroundImage: string; (image URL)
showAdvancedSettings: boolean (false by default)
reorderHandCards: boolean; (ability to rearrange cards using drag and drop)

beforeLoopActions

Array of Actions; (is performed once before game loop)

gameLoop

sequence of game actions performed in a loop until the win condition is met or the game is forced to stop.
There is the cached variable 'gameLoopIndex' : number, started with 0.

The main element is a group of actions. To organize a nested cycle actionGroups should be combined in an array.

so gameLoop is

Array of (ActionGroup or/and Array of ActionGroup) where

ActionGroup is record like: {
required fields:
name: string;
actions: Array of Actions;

optional fields:
skipCondition: Computed Payload Field or Array of Computed Payload Field. Describes the conditions under which an ActionGroup is skipped. If an Array, then it works as a logical OR with the elements of the array. Selector or Array must return TRUE to skip the action group.
turnSpectatorsToPlayers: boolean; (if 'true' and if allowSpectatorBecomePlayer in gameInitOptions is 'true' at the beginning of the action group (even if the group is skipped), the actions described in turnSpectatorToPlayerActions for spectators who have accepted the offer to become players will be performed)
turnPlayersToSpectators: boolean; (if 'true' and if allowPlayerBecomeSpectator in gameInitOptions is 'true' at the beginning of the action group (even if the group is skipped),

the actions described in turnPlayerToSpectatorActions for players who have accepted the offer to become spectators will be performed)

nextGroupNonStop: boolean; (If the game is set to autoGame = false, setting this field to false will cause the game to interrupt (pause) after executing this group of actions. Applies mainly when debugging a setup).

checkWinCondition: boolean; if 'true' after performing the group of actions will check the conditions of the end of the game (win conditions)

isGameOver: boolean; if 'true' the game will end without checking the win conditions

parallel: is used to organize the parallel execution of action groups. In this case action groups must have different names.

{

required fields:

type: string; ('independent' - parallel actions do not affect each other,
'dependent' - actions can affect the performance of other parallel actions,
'smart' - creates N (see below **qnt** option) independent parallel action groups from one ActionGroup. Each of the action group have unique value of the variable 'spaIndex' : number (saved in cache) started with 0)

optional fields:

conditions: record like (necessary for the last in the list of 'dependent' parallel action groups):

{
'actionGroup name':
{
'action result' : parallelHandler
},
…
}

(After completing a group of actions, if a handler is found by its name and the result of the last action, then it is executed. Example: { "FIRST trial vote": { "RESCIND": "stopAllVotes" } })

isLast: boolean; ('true' is necessary for the last in the list of parallel action groups. No need for 'smart' type.)

qnt: number or Computed Payload Field (is needed for type 'smart'. Number of creating parallel action groups)

}

repeat: is used to organize the successive repetitive performance of action groups. Each of the action group have unique value of the variable 'repeatIndex' : number (saved in cache) started with 0)

{

required fields:

qnt: number or Computed Payload Field (number of repetitions)

}

winCondition

record like { teamId (string): Computed Payload Field, …}. Selector must return TRUE for team victory.

postGameActions

Array of Actions;

turnSpectatorToPlayerActions

Array of Actions;

turnPlayerToSpectatorActions

Array of Actions;

Computed Payload Field

record {
    selector: string, (selector name)
    params (optional): Array of
        {
        name: string; (selector parameter name)
        type: ‘preset’ or ‘cached’ or ‘computed’
        value: any for ‘preset’ type
            cache field (string) for ‘cached’
            Computed Payload Field for ‘computed’
        }
}

Nested cycle
To organize a nested loop, groups of actions are combined into an array. You cannot create a nested loop in a nested loop. To exit a nested loop after a certain action group (the condition checks at the end of the action group), you must set the cache field **isActionLoop** to false in that action group.There is the cached variable ‘loopIndex’ : number, started with 0.